Q1 Implicit list

8 Points

Suppose your implicit list design uses both header and footer. Both have the following type (Lecture slides 30):

```
typedef struct {
  unsigned long size_and_status;
  unsigned long padding;
} header;
```

16B → 0x10

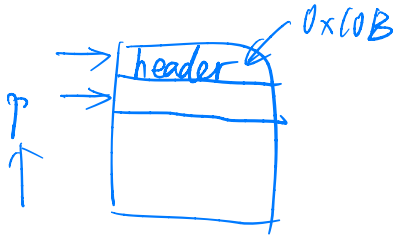
Q1.1 payload2header example

1 Point

Suppose a user invokes free(p) using pointer `p` whose value is `0x789012345670`. What is the memory address for the start of the chunk that contains the allocated space (payload) that should be freed?
(To faciliate autograding, please write your answer in hex with prefix `0x`, ignoring leading zeros and using lowercase letters)

0x789012345660

sizeof(header) = 0x10
0x…670 - 0x10 = 0x…660



Q1.2 payload2header
1 Point

`payload2header` takes as argument a pointer to the start of the payload in the chunk, and returns a pointer to the chunk's header.

```
header* payload2header(void *p)
{
    header *h;
    _____???__________;
    return h;
}
```

Which of the following C statement to use for the missing line?

- `h = (header *)p - sizeof(header);`
- `h = (header *)p - 1;`
- `h = (header *)((char *)p - sizeof(header));`
- `h = (char *)p - 1;`
- None of the above.

Q1.3 header2footer example
1 Point

$2^{10} = 2^2 \cdot 2^4 \cdot 2^4 = 0x400$

Suppose pointer variable `h` points to the beginning of a chunk and has value `0x7890123456a0`. If the total size of the chunk is 1KB (including header and footer fields), then what is the memory address for the footer of this chunk?

1KB=1024B, 1024=0x400

(To faciliate autograding, please write your answer in hex with prefix `0x`, ignoring leading zeros and using lowercase letters)

0x789012345a90

0x…6a0 + 0x400 - 0x10(footer) = 0x…a90

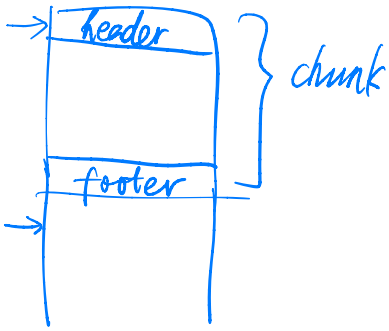
Q1.4 header2footer
1 Point

`header2footer` takes as argument a pointer to the start of the chunk, and returns a pointer to the same chunk's footer.

```
header* header2footer(header *h)
{
    header *f;
    _____???__________;
    return f;
}
```

Which of the following C statement to use for the missing line? Note that `get_size` is a helper function that returns the chunk size encoded in the header/footer field size_n_status.

- `f = h + 1;`
- `f = h - 1;`
- `f = h + get_size(h);`
- `f = (header *)((char *)h + get_size(h));`
- `f = h - get_size(h);`
- `f = (header *)((char *)h - get_size(h));`
- `f = (header *)((char *)h + get_size(h) - sizeof(header));`
- `f = (header *)((char *)h - get_size(h) + sizeof(header));`
- None of the above.



Q1.5 footer2header example
1 Point

Suppose pointer variable `f` points to the beginning of a chunk's footer and has value `0x789012345a90`. If the total size of the chunk is 1KB (including header and footer fields), then what is the memory address for the header of this chunk?

(To faciliate autograding, please write your answer in hex with prefix `0x`, ignoring leading zeros and using lowercase letters)

0x7890123456a0

See 1.3

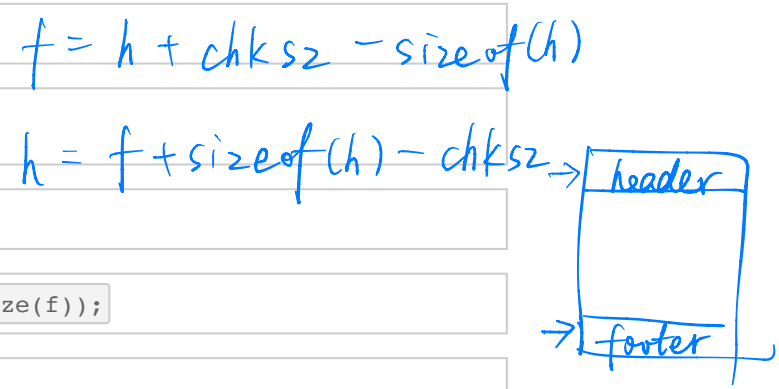
Q1.6 footer2header
1 Point

`footer2header` takes as argument a pointer to the footer of the chunk, and returns a pointer to the same chunk's header.

```
header* footer2header(header *f)
{
    header *h;
    _____???__________;
    return h;
}
```

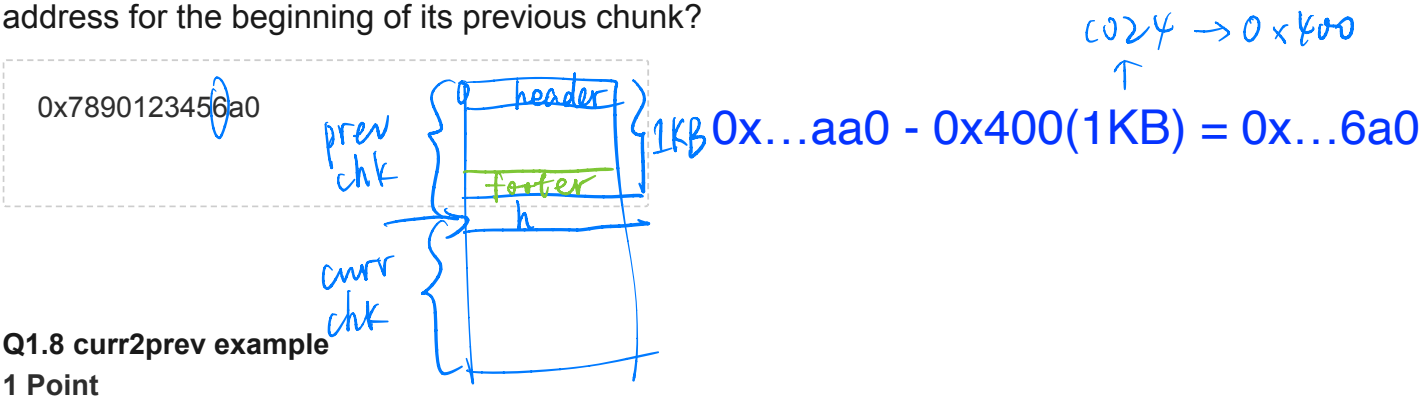
Which of the following C statement to use for the missing line `? Note that` get_size` is a helper function that returns the chunk size encoded in the header/footer field size_n_status.

- `h = f + 1;`
- `h = f - 1;`
- `h = f + get_size(f);`
- `h = (header *)((char *)f + get_size(f));`
- `h = f - get_size(f);`
- `h = (header *)((char *)f - get_size(f));`
- `h = (header *)((char *)f + sizeof(header) - get_size(f));`
- `h = (header *)((char *)f - sizeof(header) + get_size(f));`
- None of the above.



Q1.7 curr2prev example
1 Point

Suppose pointer variable `h` points to the beginning of some chunk and has value `0x789012345aa0`. Suppose this chunk has size 4KB and its previous chunk has size 1KB. What is the memory address for the beginning of its previous chunk?



Q1.8 curr2prev example
1 Point

`curr2prev` takes as argument a pointer to the current chunk's header, and returns a pointer to the previous chunk's header.

```
header* curr2prev(header *curr)
{
    header *prev_footer;
    _____???__________;
    return footer2header(prev_footer);
}
```

Which of the following C statement to use for the missing line? Note that `footer2header` is the helper function that returns a pointer to the chunk's header given a pointer to the same chunk's footer.

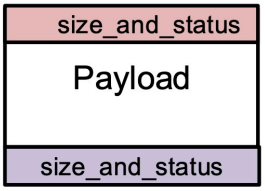
- `prev_footer = curr -1 ;`
- `prev_footer = curr - sizeof(header);`
- `prev_footer = (header *)((char *)curr - sizeof(header));`
- `prev_footer = curr - 2;`
- `prev_footer = curr - 2*sizeof(header);`
- `prev_footer = (header *)((char *)curr - 2*sizeof(header));`
- None of the above.

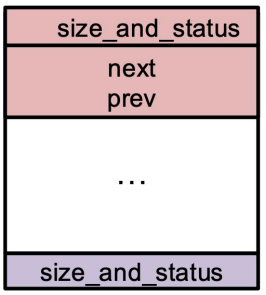
Q2 Explicit list

1 Point

Which of the following statements are true about explicit list?

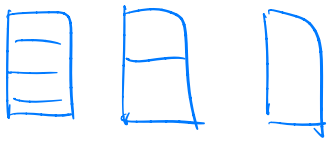
- The explicit list design explicitly chains together all chunks of the heap into a linked list.
- The explicit list design explicitly only chains together all free chunks of the heap into a linked list.
- The explicit list design incurs more memory overhead than the implicit list design because it uses extra space in the header to store the next/prev fields.
- `malloc(...)` in the explicit list design is faster than that of implicit list because it does not need to scan over allocated chunks.
- `free(...)` in the explicit list design is faster than that of implicit list because it does not need to scan over allocated chunks.





Q3 Buddy system

1 Point



Which of the following statements are true about the buddy system?

- [x] All chunks have sizes that are powers-of-2.
- [ ] During `free(...)`, coalescing only happens once by merging the freed chunk with its buddy of the same size if the buddy is free.
- [x] During `free(...)`, coalescing is done recursively by repeatedly merging the freed chunk with its buddy of the same size and repeating the merge process for the resulting larger free chunk until its buddy is no longer free.
- [x] The design maintains multiple free lists each of which contains free chunks of the same (powers-of-2) size.
- [ ] The design maintains a single free list containing all free chunks.